

Mission 005 Backend Handoff

Status

The backend implementation is sound enough for QA and frontend work to proceed against the current contract.

What was done

- Added the canonical agent write path at ``POST /api/agent-events``.
- Added a simple canonical write path at ``POST /api/agent-events`` that validates payload shape and writes to Neon without a write secret.
- Added Neon-backed persistence for canonical mission state and mission events.
- Preserved latest-mission reads, freshness/lag metadata, and updated-at visibility in the read service.
- Added the Neon migration for the canonical tables and receipt storage.

Relevant files

- ``src/app/api/agent-events/route.ts``
- ``src/lib/agent-event-auth.ts``
- ``src/lib/dashboard-store.ts``
- ``src/lib/dashboard-service.ts``
- ``db/migrations/001_mission_005_neon.sql``

Verification

- ``pnpm test`` ■
- 8 test files passed
- 20 tests passed
- 5 todo tests
- ``pnpm typecheck`` ■

Backend caveats / follow-ups

- There are still no dedicated tests for ``POST /api/agent-events`` or the Neon write transaction path; current verification is mostly read-path coverage plus typecheck.
- ``X-Agents-Vis-Key-Id`` is no longer part of the v1 write path; the route now validates payload shape and persists directly to Neon.
- ``parallelOrder`` and ``parallelSize`` are only checked as finite numbers; they are not explicitly constrained to integers/non-negative values in the route.
- The JSON data source fallback still exists when DB env vars are absent; production/preview must supply Neon envs so the canonical DB remains the active source.
- Test runs emit a non-blocking Neon deprecation warning about ``fetchConnectionCache``.

Next dependency

- QA can now validate the simple write endpoint against Neon and confirm that dashboard reads reflect writes end-to-end.
- Frontend work can proceed assuming the read contracts in `src/lib/dashboard-types.ts` remain stable.